Boop Car Report

Elise McDonald
Shealyn Keelen

Date of Report: April 24, 2026

Contents

1 Introduction and Overview **1**

2 System Overview **1**
2.1 Sensing Elements 2
2.2 Movement Control 2

3 Hardware Setup **3**

4 Code **3**

5 Performance Overview **8**

6 Conclusions **10**

7 Full Written Code **10**

1 Introduction and Overview

The objective of this project was to design and implement an autonomous car capable of identifying an object (Sasquatch) within a 2 meter diameter circle. The system integrates multiple components, everything included in the Lafvin car kit: including DC motors with a motor control shield, an ultrasonic distance sensor for object detection, and a line sensor for boundary detection. These elements are controlled using the Arduino microcontroller, which processes sensor inputs and executes movement decisions.

The problem addressed in this project is autonomous navigation within a constrained arena. Specifically, the car must locate a target object placed within a circular boundary and move toward it within a 2 minute time frame. This requires the system to continuously sense its surroundings, make decisions based on real-time data, and adjust its motion accordingly.

At a high level, the system operates by first moving forward and then performing a 360° scan using the ultrasonic sensor to determine the location of the closest object. Once the object is identified, the car rotates to align itself with the target and proceeds forward. During movement, the line sensor continuously monitors for boundary conditions. If a boundary is detected, the car reverses and restarts its search process. The system repeats this cycle until the object is reached within a predefined stopping distance, at which point the car stops.

2 System Overview

The system consists of three main components: sensing, processing, and actuation. The ultrasonic sensor and line sensor provide input data about the environment, such as how close an object is from the car and if the car has passed over a black line or not. This is then sent to the Arduino microcontroller for processing. Based on this information, the Arduino determines the appropriate movement and sends control signals to the motor driver, which drives the four DC motors.

A flowchart of the system's decision making process is shown in Figure 1. The system begins by moving forward and then performs a 360° sweep to detect the closest object using the ultrasonic sensor. Once the closest object is identified, the car rotates to face it and begins moving forward. During this process, the line sensor continuously checks for boundary conditions. If a boundary is detected, the car backs up and restarts the scanning process. If the object is reached within a predefined distance, the system stops.

Overall, the system follows an input, processing, output structure. Sensor data (distance and line detection) serve as inputs, the Arduino executes decision-making logic as the processing stage, and motor commands act as the output that controls the car's motion.

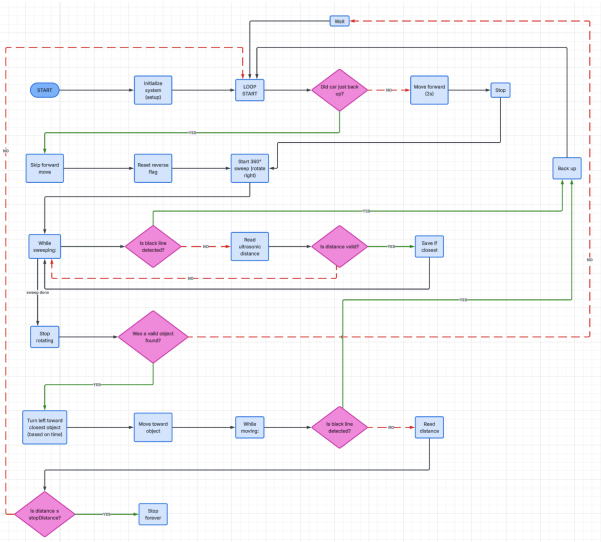

Figure 1: Flow Chart Scheme

2.1 Sensing Elements

The car used an ultrasonic distance sensor and an IR line tracking sensor to interact with the arena. The ultrasonic sensor was connected to analog pins A0 and A1 and was used to detect objects in front of the car. During operation, the car performed a 360° scan while continuously recording distance measurements. The shortest valid distance was saved as the target object location. This allowed the car to estimate which direction the object was located before driving toward it. The line tracking sensor was connected to pin A2 and was used to detect the black electrical tape marking the boundary of the arena. Testing showed that the sensor output was HIGH when placed over the black line, so the code checked for this condition using "digitalRead(lineSensorPin) == HIGH" which can be review in figure 5. If the black line was detected, the car stopped and backed up toward the center before restarting the scan. This prevented the car from leaving the arena.

2.2 Movement Control

The car's movement was controlled using the L293D motor shield and four DC motors. The motor shield pins controlled motor speed using PWM and motor direction. Movement functions were created for forward, backward, stopping, and rotating right or left. Forward and backward motion were achieved by driving all four motors in the same direction, while rotation was created by driving the motors on each side in opposite directions, causing the car to pivot in place. The speed of the car was adjusted by varying the PWM values (0-255) sent to each motor, allowing for smoother and more controlled movement during scanning and driving. These functions were repeatedly called throughout the program to switch between behaviors such as scanning, repositioning, and approaching the target. At the start of the program, the car moved forward for 2 seconds to position itself in the center of the 2 m diameter arena. It then stopped and performed a 360° rotation to scan for nearby objects. Once the closest object was identified, the car rotated to face that direction and moved toward it. During this process, the car continuously monitored the boundary. If the black line was detected, it stopped, backed up toward the center,

and restarted the scanning routine.

3 Hardware Setup

The sensing subsystem consists primarily of an ultrasonic distance sensor and a line detection sensor. The ultrasonic sensor is used to measure the distance to nearby objects and is connected to the Arduino through two pins: a trigger pin and an echo pin. The line sensor is connected to a digital input pin and is positioned underneath the car to detect the boundary line of the arena.

The ultrasonic sensor is mounted at the front of the vehicle to allow for forward-facing distance measurements during both scanning and movement. The line sensor is mounted near the base of the car, facing downward, to detect contrast between the floor and the boundary tape. Proper placement of both sensors is critical to ensure accurate readings and reliable system behavior. The hardware used to construct this system is listed below. The connection diagram for this system is shown in Figure 2.

1. Line Tracking Module
2. Arduino Microcontroller
3. Lafvin Car Kit
4. 2x 3.7 V Li-ion Batteries

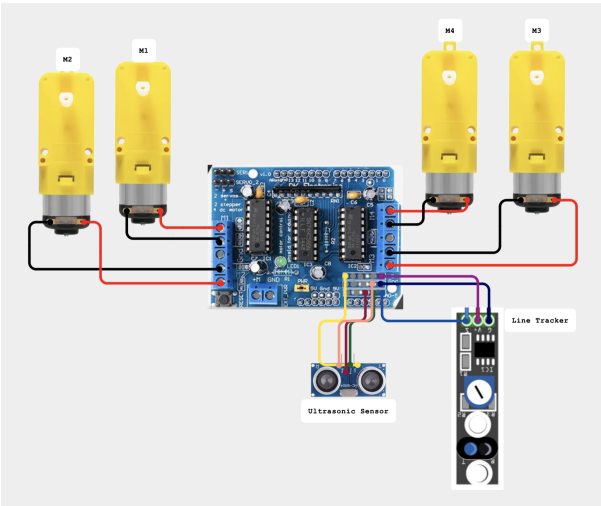

Figure 2: Connection Scheme

4 Code

The code for the system was written in the Arduino IDE. The code started by calling in the library for the ultrasonic sensor. Then each of the motor pins were defined and the predefined codes were assigned to their corresponding movement direction. The PWM speeds for each motor were also defined at a value of 110. This section of the code is shown in Figure 3.

```
#include <Ultrasonic.h>

//defining the pins of the motor
const int PWM2A = 11;
const int PWM2B = 3;
const int PWM0A = 6;
const int PWM0B = 5;
const int DIR_CLK = 4;
const int DIR_EN = 7;
const int DATA = 8;
const int DIR_LATCH = 12;

//Predefined codes for movement directions
const int Move_Forward  = 39;
const int Move_Backward = 216;
const int Right_Rotate  = 149;
const int Left_Rotate   = 106;
const int Stop          = 0;

//defining the PWM speeds of each motor
int Speed1 = 110;
int Speed2 = 110;
int Speed3 = 110;
int Speed4 = 110;
```

Figure 3: Start of System Code

Then the pins for the ultrasonic sensor and the line sensor were defined. A variable called "justBackedUp" was created to check if the system had just backed up, indicating that the system's main loop would be repeating. Several parameters were then defined for later use. These values included the time for the car to make a full scan, the time for the car to drive to the center of the circle, as well as a minimum and maximum valid distance for ultrasonic sensing. This section of the code is shown in Figure 4.

```
//defining the ultrasonic sensor
//A0 is the trigger pin
//A1 is the echo pin
Ultrasonic ultrasonic(A0, A1);

//defining pin of the line sensor
const int lineSensorPin = A2;

//creating a variable to check whether the car has recently backed up
bool justBackedUp = false;

//defining timing parameters
const unsigned long sweepTime = 3000; //time to make full scan
const int minValidDistance = 2; //setting min distancce
const int maxValidDistance = 200; //setting max distance
const unsigned long forwardCheckDelay = 10; //how often to check distance reading
const unsigned long backUpToCenterTime = 2000; //time to reverse when line detected
const unsigned long turnBackCorrection = 180; //correction for aiming back at the object
```

Figure 4: Continued Setup for System's Code

Several functions were then defined for later use. The "motor" function was included from use in previous projects to control the speed of each of the motors at once. This function used the analogWrite command to assign a PWM value for speed to each of the motor pins. This function served as the foundation for all of the systems movement. The

”getDistanceCM” function was created to read the ultrasonic sensor and return a distance measurement. This function filtered out any values that fell outside an acceptable range, ensuring that the returned measurement was valid. The ”blackLineDetected” function was also created for the line sensor in order to detect whether the car had driven over a black line. This section of the code is shown in Figure 5.

```
//function to control each of the motors at once
void Motor(int Dir, int Speed1, int Speed2, int Speed3, int Speed4) {
  analogWrite(PWM2A, Speed1);   //writing PWM value to motor 1
  analogWrite(PWM2B, Speed2);   //writing PWM value to motor 2
  analogWrite(PWM0A, Speed3);   //writing PWM value to motor 3
  analogWrite(PWM0B, Speed4);   //writing PWM value to motor 4

  digitalWrite(DIR_LATCH, LOW);
  shiftOut(DATA, DIR_CLK, MSBFIRST, Dir);
  digitalWrite(DIR_LATCH, HIGH);
}

//function to read the distance
int getDistanceCM() {
  int d = ultrasonic.read(); //reads ultrasonic distance
  if (d < minValidDistance || d > maxValidDistance) return -1;
  return d;
}

//function for line detection
bool blackLineDetected() {
  return digitalRead(lineSensorPin) == HIGH; //returns true if a line is detected
}
```

Figure 5: Functions for System Code

Next, several functions were written as shortcuts for common movements, such as stopping the car, turning left, and moving forward. These functions called the previously defined motor control function as well as the predefined direction values to execute their desired motions. This section of the code is shown in Figure 6.

```
//shortcut function to stop car
void stopCar() {
  Motor(Stop, 0, 0, 0, 0);
}

//shortcut function to rotate right
void rotateRight() {
  Motor(Right_Rotate, Speed1, Speed2, Speed3, Speed4);
}

//shortcut function to rotate left
void rotateLeft() {
  Motor(Left_Rotate, Speed1, Speed2, Speed3, Speed4);
}

//shortcut function to moveforwards
void moveForward() {
  Motor(Move_Forward, Speed1, Speed2, Speed3, Speed4);
}

//shortcut function to move back
void moveBackward() {
  Motor(Move_Backward, Speed1, Speed2, Speed3, Speed4);
}
```

Figure 6: Movement Shortcut Functions

A function was then written for the situation where the car crosses the boundary line. The purpose of this function was to have the car back up to the center of the circle to restart the scanning process in the event that the system missed the target. This was achieved by commanding the car to stop and move backwards to the center of the circle. After reversing, the ”justBackedUp” variable was updated to indicate that the backup was completed. Another function was created to command the system to rotate towards

the closest detected object. This was accomplished by recording the time at which the closest object was detected during the scanning phase. This time value was then used to calculate how long the car should rotate in the opposite direction to end up facing the closest detected object. A correction factor was included in an attempt to improve accuracy. This section of the code is shown in Figure 7.

```
//function for when line is detected
void backUpAndRestart() {
  stopCar(); //car is stopped
  delay(200);

  moveBackward(); //car moves backwards
  delay(backUpToCenterTime); //car will move back to the center

  stopCar(); //car is stopped again in the center
  delay(300);

  justBackedUp = true; //setting variable back to true
}

//
void turnToClosestObject(unsigned long bestTime) {
  unsigned long turnBackTime = 0; //variable to store how long the car should turn back


  if ((sweepTime - bestTime) > turnBackCorrection) {
    //calculating how long to rotate to face closest object
    //includes correction time
    turnBackTime = (sweepTime - bestTime) + turnBackCorrection;
  }

  rotateLeft(); //turns the car left
  delay(turnBackTime); //continue rotating for calculated tme

  stopCar();  //stops car after turning
  delay(500);
}
```

Figure 7: Functions for System

In the setup section of the code, each of the motor control pins were configured as outputs, and the line sensor pin was configured as an input. Serial communication was then initialized and the motor driver was enabled. The car was commanded to stop for a short time to ensure a stable starting position before entering the main loop. This section of the code is shown in Figure 8.

```
//Setup
void setup() {
  //setting all motor control pins as outputs
  pinMode(DIR_CLK, OUTPUT);
  pinMode(DATA, OUTPUT);
  pinMode(DIR_EN, OUTPUT);
  pinMode(DIR_LATCH, OUTPUT);
  pinMode(PWM0B, OUTPUT);
  pinMode(PWM0A, OUTPUT);
  pinMode(PWM2A, OUTPUT);
  pinMode(PWM2B, OUTPUT);

  //setting line sensor as an input
  pinMode(lineSensorPin, INPUT);

  //initiallizing communication
  Serial.begin(9600);

  //enabeling motor driver
  digitalWrite(DIR_EN, LOW);

  stopCar();  //stopping car before entering main loop
  delay(1000);
}
```

Figure 8: Setup for System Code

The main loop of the code was then written. At the start of the loop several variables were initialized to track the closest detected object as well as the time at which the closest object was observed. Then, if the car has not just recently backed up, the car would move forward into the center of the circle and stop. If the car had just recently backed up, the forward motion was skipped. The car then began a 360 degree scan by rotating to the right and continually reading from the ultrasonic sensor. During the scan, the system would keep track of the closest detected object and record the time at which the closest object was detected. If a line was detected by the line sensor at any point during the scan, the car would immediately back up and restart the loop. After the scan was completed, the system would check if a valid object was detected. If a valid object was detected, the car used the tracked information to rotate to face the closest detected object. Once aligned with the object, the car would move forward towards it. While moving, the car continually checked to see if a line was detected, indicating that it missed the target. If a line was detected, the car would backup and restart the loop again. This section of the code is shown in Figures 9 and 10.

```
void loop() {
  int closestDistance = 9999; //initialize variable to store closest distance found
  unsigned long bestTime = 0; //variable to store time when closest object detected

  //if the car has not just backed up
  if (!justBackedUp) {
    moveForward();
    delay(2000);  //car will move forward for 2 seconds
    stopCar();    //car will stop for a short period of time
    delay(300);
  } else {
    //if the car has just backed up, reset the variable for the next loop
    justBackedUp = false;
  }

  //record starting time of the scan
  unsigned long startTime = millis();

  //rotate right to perform scan
  rotateRight();

  //continue rotating for the full scan
  while (millis() - startTime < sweepTime) {
    //if a line is detected during the scan
    if (blackLineDetected()) {
      backUpAndRestart();   //execute backup function
      return;               //restart next loop
    }

    //read distance from ultrasonic sensor
    int distance = getDistanceCM();

    //if the distance reading is valid
    if (distance != -1) {
```

Figure 9: System's Main Loop

```
      //check if this is the closest detected object
      if (distance < closestDistance) {
        closestDistance = distance;       //update closest distance
        bestTime = millis() - startTime;  //record when it was detected
      }
    }

    delay(100);
  }

  //stop rotating after scan is complete
  stopCar();
  delay(500);

  //turn toward the closest detected object
  turnToClosestObject(bestTime);

  //begin movement toward the object
  while (true) {

    //if a line is detected while moving forward
    if (blackLineDetected()) {
      backUpAndRestart(); //execute backup function
      return;             //restart loop
    }

    //move forward to the oject
    moveForward();
    delay(10);
  }
}
```

Figure 10: Main Loop Continued

5 Performance Overview

During the official Boop competition, the system successfully located the object in all 3 out of 3 attempts. Although it was not the fastest, with an average time of approximately 25 seconds, the system performed reliably and completed the task successfully every attempt. Images of the system locating and "booping" the target are shown in Figures 11 and 12.

Figure 11: System Locating Target

Figure 12: System Hitting Target

The primary strength of the system was its consistency. It was able to locate the object in under 25 seconds in each attempt, meeting the competition requirements. The use of both the ultrasonic sensor and line sensor allowed for reliable navigation and boundary detection, preventing the car from leaving the arena.

However, there are several limitations that impacted performance. The system was limited by speed and overall efficiency, as well as some minor inaccuracies in object targeting. Delays within the code and conservative movement speeds contributed to longer completion times. Additionally, slight drifting during movement reduced accuracy and required correction.

There are multiple areas for improvement. Increasing motor speed and reducing delays in the code would improve overall efficiency. Mechanical adjustments to reduce drifting would also enhance accuracy. Additionally, implementing a second scan before committing to movement could improve object targeting and reduce time lost due to

misalignment. These improvements would likely result in faster and more precise performance.

6 Conclusions

This project helped to develop a much stronger understanding how a system containing multiple sensors can be created to accomplish autonomous behavior with a robotic car. Using the ultrasonic sensor in combination with the line tracking module allowed us to gain experience with how a microcontroller can be used for both object and boundary detection, allowing for the system to function safely within a defined environment. The most important takeaway from this project was learning the importance of testing the system and making adjustments as needed to ensure a functional final product.

One of the largest challenges with this project was dealing with the discrepancies between the theoretical behavior and the real-world performance of the car. For example, the movement of the car was not perfectly precise, as there was a small amount of drift in its rotational movement. This often lead to inaccuracies when attempting to track ad align with the object in the circle. As a result, small adjustments had to be made to the code so the system could reliably face the object without overshooting or undershooting. This was addressed through slight timing adjustments in the code, as well as adding logic to handle situations where the system would miss the target and reach the boundary. In these cases, the system would reverse to the center of the circle and restart its scanning process to attempt to reach the target again within the 2-minute time limit. This allowed our system to successfully reach the target in all three attempts within the time constraint.

7 Full Written Code

```
#include <Ultrasonic.h>

//defining the pins of the motor
const int PWM2A = 11;
const int PWM2B = 3;
const int PWM0A = 6;
const int PWM0B = 5;
const int DIR_CLK = 4;
const int DIR_EN = 7;
const int DATA = 8;
const int DIR_LATCH = 12;

//Predefined codes for movement directions
const int Move_Forward  = 39;
const int Move_Backward = 216;
const int Right_Rotate  = 149;
const int Left_Rotate   = 106;
const int Stop          = 0;

//defining the PWM speeds of each motor
int Speed1 = 110;
```

```
int Speed2 = 110;
int Speed3 = 110;
int Speed4 = 110;

//defining the ultrasonic sensor
//A0 is the trigger pin
//A1 is the echo pin
Ultrasonic ultrasonic(A0, A1);

//defining pin of the line sensor
const int lineSensorPin = A2;

//creating a variable to check whether the car has recently backed up
bool justBackedUp = false;

//defining timing parameters
const unsigned long sweepTime = 3000; //time to make full scan
const int minValidDistance = 2; //setting min distancce
const int maxValidDistance = 200; //setting max distance
const unsigned long backUpToCenterTime = 2000; //time to reverse when line detected
const unsigned long turnBackCorrection = 180; //correction for aiming back at the obj

//function to control each of the motors at once
void Motor(int Dir, int Speed1, int Speed2, int Speed3, int Speed4) {
  analogWrite(PWM2A, Speed1);   //writing PWM value to motor 1
  analogWrite(PWM2B, Speed2);   //writing PWM value to motor 2
  analogWrite(PWM0A, Speed3);   //writing PWM value to motor 3
  analogWrite(PWM0B, Speed4);   //writing PWM value to motor 4

  digitalWrite(DIR_LATCH, LOW);
  shiftOut(DATA, DIR_CLK, MSBFIRST, Dir);
  digitalWrite(DIR_LATCH, HIGH);
}

//function to read the distance
int getDistanceCM() {
  int d = ultrasonic.read(); //reads ultrasonic distance
  if (d < minValidDistance || d > maxValidDistance) return -1;
  return d;
}

//function for line detection
bool blackLineDetected() {
  return digitalRead(lineSensorPin) == HIGH; //returns true if a line is detected
}


//shortcut function to stop car
```

```
void stopCar() {
  Motor(Stop, 0, 0, 0, 0);
}

//shortcut function to rotate right
void rotateRight() {
  Motor(Right_Rotate, Speed1, Speed2, Speed3, Speed4);
}

//shortcut function to rotate left
void rotateLeft() {
  Motor(Left_Rotate, Speed1, Speed2, Speed3, Speed4);
}

//shortcut function to moveforwards
void moveForward() {
  Motor(Move_Forward, Speed1, Speed2, Speed3, Speed4);
}

//shortcut function to move back
void moveBackward() {
  Motor(Move_Backward, Speed1, Speed2, Speed3, Speed4);
}

//function for when line is detected
void backUpAndRestart() {
  stopCar(); //car is stopped
  delay(200);

  moveBackward(); //car moves backwards
  delay(backUpToCenterTime); //car will move back to the center

  stopCar(); //car is stopped again in the center
  delay(300);


  justBackedUp = true; //setting variable back to true
}

//
void turnToClosestObject(unsigned long bestTime) {
  unsigned long turnBackTime = 0; //variable to store how long the car should turn ba


  if ((sweepTime - bestTime) > turnBackCorrection) {
    //calculating how long to rotate to face closest object
    //includes correction time
    turnBackTime = (sweepTime - bestTime) + turnBackCorrection;
```

```
  }

  rotateLeft(); //turns the car left
  delay(turnBackTime); //continue rotating for calculated tme

  stopCar();  //stops car after turning
  delay(500);
}

//Setup
void setup() {
  //setting all motor control pins as outputs
  pinMode(DIR_CLK, OUTPUT);
  pinMode(DATA, OUTPUT);
  pinMode(DIR_EN, OUTPUT);
  pinMode(DIR_LATCH, OUTPUT);
  pinMode(PWM0B, OUTPUT);
  pinMode(PWM0A, OUTPUT);
  pinMode(PWM2A, OUTPUT);
  pinMode(PWM2B, OUTPUT);

  //setting line sensor as an input
  pinMode(lineSensorPin, INPUT);

  //initiallizing communication
  Serial.begin(9600);

  //enabeling motor driver
  digitalWrite(DIR_EN, LOW);

  stopCar();  //stopping car before entering main loop
  delay(1000);
}

void loop() {
  int closestDistance = 9999; //initialize variable to store closest distance found
  unsigned long bestTime = 0; //variable to store time when closest object detected

  //if the car has not just backed up
  if (!justBackedUp) {
    moveForward();
    delay(2000);  //car will move forward for 2 seconds
    stopCar();    //car will stop for a short period of time
    delay(300);
  } else {
    //if the car has just backed up, reset the variable for the next loop
    justBackedUp = false;
  }
```

```
//record starting time of the scan
unsigned long startTime = millis();

//rotate right to perform scan
rotateRight();

//continue rotating for the full scan
while (millis() - startTime < sweepTime) {
  //if a line is detected during the scan
  if (blackLineDetected()) {
    backUpAndRestart();   //execute backup function
    return;               //restart next loop
  }

  //read distance from ultrasonic sensor
  int distance = getDistanceCM();

  //if the distance reading is valid
  if (distance != -1) {

    //check if this is the closest detected object
    if (distance < closestDistance) {
      closestDistance = distance;       //update closest distance
      bestTime = millis() - startTime;  //record when it was detected
    }
  }

  delay(100);
}

//stop rotating after scan is complete
stopCar();
delay(500);

//turn toward the closest detected object
turnToClosestObject(bestTime);

//begin movement toward the object
while (true) {

  //if a line is detected while moving forward
  if (blackLineDetected()) {
    backUpAndRestart(); //execute backup function
    return;             //restart loop
  }

  //move forward to the oject
```

```
    moveForward();
    delay(10);
  }
}
```